

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	RAGAVI S	Reg. No.:	192565027
Section / Batch:	SLOT A	Date:	11-08-2026

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Analyze a processor datapath that performs register transfers and logical operations such as XOR, NOT, and comparison, and examine the control signals and execution sequence required for each operation.
2. Develop a bus-based system for transferring data between the processor, memory, and I/O devices, and analyze how bus width, transfer rate, and bus contention influence system performance.
3. Simulate a pipelined processor with arithmetic, memory, and branch instructions, and evaluate the effects of pipeline depth on execution time, throughput, and overall speedup.
4. Construct a pipeline containing instruction dependencies and branch conditions, and analyze the performance impact of data and control hazards using stalling, forwarding, and branch prediction techniques.
5. Develop a superscalar processor model that supports multiple execution units and out-of-order instruction processing, and evaluate how instruction scheduling and resource allocation improve instruction throughput.

Code of Conduct

*I **RAGAVI S / 192565027** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: RAGAVI S

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

QUESTION 1: Processor Datapath & Control Sequence Analysis

1.1 Theoretical Framework & Architectural Background

A processor datapath consists of hardware units (registers, Arithmetic Logic Unit (ALU), internal buses, and data multiplexers) designed to perform data processing tasks. Operands flow through register files, undergo transformation inside the ALU, and write back into specified registers under the exact coordination of control signals generated by the Control Unit (CU). In a typical RISC-like single-cycle or multi-cycle datapath, logical and arithmetic execution rely on dedicated control signals to route signals, select operational modes, and maintain synchronization.

1.2 Datapath Architecture & Control Signals

To perform register transfers and basic logical/comparison operations, the datapath uses specific control lines:

- **RegDst / WriteRegister:** Selects which instruction field determines the destination register address.
- **ALUSrc:** Chooses between the second register operand read port or an immediate zero-extended/sign-extended value.
- **ALU_Op [2:0]:** A multi-bit bus decoded by the ALU controller to dictate the specific operation (e.g., 000=AND, 001=OR, 010=XOR, 011=NOT, 100=ADD, 101=SUB/COMPARE).
- **RegWrite:** Enable signal. When high (1) at the rising clock edge, writes the ALU result into the destination register.
- **MemToReg:** Directs either the ALU output or memory output to the register write port.
- **Status Flags (Zero, Sign, Overflow, Carry):** Hardware bit indicators set dynamically by the ALU status register to record computational conditions.

1.3 Step-by-Step Execution Sequence for Operations

A. XOR Operation (Destination = R1 XOR R2)

1. Instruction Fetch (IF): The instruction encoding `XOR R3, R1, R2` is retrieved from memory using the Program Counter (PC).
2. Instruction Decode & Read (ID): Read ports load values from source registers R1 and R2 onto internal buses A and B.
3. Execute (EX): Control signal ALUSrc is set to 0. Control unit asserts `ALU\_Op = XOR`. The ALU performs bitwise XOR gate evaluation across all bit-widths concurrently.
4. Write Back (WB): MemToReg is set to 0, RegWrite is set to 1. The result on the ALU output bus is latched into destination register R3 at the next active clock edge.

B. NOT Operation (Destination = NOT R1)

1. Instruction Fetch & Decode: Read source register R1 onto bus A.
2. Execute: Set ALUSrc to 0 (or select a pseudo-operand depending on implementation). Assert `ALU\_Op = NOT`. Internal inverter arrays invert each bit of input register R1.
3. Write Back: RegWrite is asserted (1), writing the inverted bitvector into the destination register.

C. Comparison Operation (Compare R1 and R2)

1. Instruction Fetch & Decode: Load register values R1 and R2.
2. Execute: Assert `ALU\_Op = SUBTRACT` (R1 - R2). The ALU computes the arithmetic difference. Crucially, `RegWrite` is held at 0 (disabled) so no register value is altered.
3. Flag Update: If $(R1 - R2) == 0$, the Zero Flag (Z) is set to 1. If $(R1 - R2) < 0$, the Sign Flag (S) or Negative Flag (N) is set to 1. These flags are stored in the Processor Status Register (PSR) for conditional branching.

1.4 Micro-Operation & Control Signal Summary Table

Operation	ALUSrc	ALU_Op	RegWrite	Flag Updates
XOR R3, R1, R2	0 (Reg)	010 (XOR)	1 (Write Enabled)	Zero, Sign
NOT R2, R1	0 (Reg)	011 (NOT)	1 (Write Enabled)	Zero, Sign
CMP R1, R2	0 (Reg)	101 (SUB)	0 (Write Disabled)	Zero=1 if R1=R2, Sign=1 if R1<R2
XORI R1, R1, 0x0F	1 (Imm)	010 (XOR)	1 (Write Enabled)	Zero, Sign

1.5 Performance Analysis & Datapath Optimization

The primary bottleneck in single-cycle datapath logical execution is propagation delay through multiplexers and the ALU logic gates. By optimizing ALU gate depth (e.g., utilizing parallel prefix logic for bitwise operations and fast carry-lookahead structures for comparisons), critical path latency is reduced, allowing higher maximum operational frequencies.

QUESTION 2: Bus-Based Interconnection System & Performance Factors

2.1 System Architecture & Bus Infrastructure

Modern computer systems require standard communication pathways to transfer instruction, memory data, and control status among the Processor, Main Memory, and I/O Peripherals. Bus architecture consists of three principal logical buses:

- **Data Bus:** Bi-directional pathway carrying binary instruction words and data values.
- **Address Bus:** Uni-directional or bi-directional bus specifying exact physical memory addresses or I/O device ports.
- **Control Bus:** Signals coordinating timing and device access (e.g., Read/Write enables, Memory Request, Bus Request, Bus Grant, Interrupt Requests).

2.2 Deep Dive into Bus Performance Determinants

A. Impact of Bus Width

Bus width defines the number of parallel physical lines (conductors). A 32-bit data bus transfers 4 bytes per clock cycle, whereas a 64-bit data bus transfers 8 bytes per cycle. If a processor fetches a 64-bit double-precision floating-point number over a 32-bit bus, it requires 2 bus cycles; over a 64-bit bus, it completes in a single cycle, effectively doubling data throughput.

B. Impact of Transfer Rate & Clock Frequency

Transfer rate is governed by the bus clock frequency (Hz) and data transfer techniques (e.g., Single Data Rate vs. Double Data Rate). Transfer Rate is quantified by:

$$\text{Throughput (Bytes/sec)} = \text{Bus Width (Bytes)} \times \text{Clock Frequency (Hz)} \times \text{Transfers per Cycle}$$

Increasing clock frequency reduces transaction latency, but high frequencies introduce physical design challenges such as signal attenuation, clock skew, and electromagnetic interference (EMI).

C. Bus Contention & Arbitration Schemes

Bus contention occurs when multiple master devices (e.g., Processor, Direct Memory Access (DMA) controllers, PCI express endpoints) simultaneously request control of the shared bus. Unresolved contention leads to data collisions and system stalls. Arbitration mechanisms resolve these conflicts:

- **Centralized Daisy-Chaining:** Low hardware cost; devices are daisy-chained along a grant line. Priority decreases with physical distance from the arbiter. Prone to starvation of downstream devices.
- **Centralized Parallel Arbitration:** Each device has dedicated Bus Request (BR) and Bus Grant (BG) lines to a central arbiter. The arbiter uses Fair Round-Robin or Fixed Priority algorithms, eliminating starvation.
- **Distributed Arbitration:** Devices independently negotiate access using shared priority lines, ideal for high-speed multi-master system buses.

2.3 Quantitative System Performance Comparison

Bus Type	Bus Width (bits)	Clock Rate (MHz)	Arbitration Scheme	Max Throughput (MB/s)
Legacy System Bus	32 bit	33 MHz	Daisy Chain	132 MB/s
High-Speed System Bus	64 bit	100 MHz	Parallel Fixed Priority	800 MB/s
Modern Interconnect (DDR)	64 bit	400 MHz (DDR)	Parallel Round-Robin	6,400 MB/s

2.4 System Trade-off Analysis

While expanding bus width boosts peak bandwidth, it significantly increases pin count, substrate layout complexity, and power consumption. Modern architectures address these limitations by transitioning from wide, shared parallel buses to high-speed point-to-point serial packet-based interfaces (such as PCIe and Ultra Path Interconnect).

QUESTION 3: Pipelined Processor Simulation & Pipeline Depth Evaluation

3.1 Principles of Instruction Pipelining

Instruction pipelining is an implementation technique where multiple instructions are overlapped in execution. A standard classical RISC pipeline consists of 5 stages:

- **1. Instruction Fetch (IF):** Read instruction from instruction cache; increment PC.
- **2. Instruction Decode (ID):** Read source registers; decode opcode and control bits.
- **3. Execute (EX):** Perform ALU operation, calculate memory address, or evaluate branch condition.
- **4. Memory Access (MEM):** Read/write data from/to data cache (if load/store).
- **5. Write Back (WB):** Write computed result back into register file.

3.2 Effects of Pipeline Depth on Execution Metrics

A. Execution Time & Clock Period

In a non-pipelined processor, the total instruction latency $T_{\text{non-pipe}}$ equals the sum of delays across all functional blocks. In an k -stage pipeline, the clock cycle time T_{clock} is bounded by the delay of the slowest stage (t_{max}) plus latch/register setup delay (t_{latch}):

$$T_{\text{clock}} = \max(t_1, t_2, \dots, t_k) + t_{\text{latch}}$$

Increasing depth k breaks processing down into finer sub-operations, reducing t_{max} and allowing higher clock frequencies.

B. Throughput & Theoretical Speedup

Instruction Throughput measures completed instructions per second. Under ideal conditions (no pipeline stalls or overheads), the Speedup (S_k) achieved by a k -stage pipeline over a non-pipelined system executing N instructions is:

$$S_k = \frac{N \cdot k \cdot T_{\text{clock}}}{(k + N - 1) \cdot T_{\text{clock}}} \rightarrow \infty \text{ as } N \rightarrow \infty$$

Thus, maximum speedup theoretically approaches the number of pipeline stages k .

C. The Deep Pipeline Penalty (Branch Misprediction & Latency Overheads)

As pipeline depth becomes extreme (superpipelining, e.g., 20–30 stages), setup times (t_{latch}) consume a larger percentage of the clock cycle. Furthermore, when branch mispredictions occur, all $k-1$ in-flight instructions prior to the branch target must be flushed, causing a penalty of $k-1$ wasted clock cycles. Consequently, past an optimal depth threshold, overall execution performance declines.

3.3 Simulation Performance Comparison Across Pipeline Depths

Pipeline Configuration	Clock Frequency	Ideal CPI	Misprediction Penalty	Real-World Execution Time (10^6 Inst)
Non-Pipelined (1 Stage)	500 MHz	1.00	0 Cycles	2.00 ms
Standard Pipeline (5 Stage)	2.0 GHz	1.15	4 Cycles	0.58 ms
Deep Pipeline (12 Stage)	3.8 GHz	1.45	11 Cycles	0.38 ms
Ultra-Deep Pipeline (31 Stage)	4.5 GHz	2.30	30 Cycles	0.51 ms

3.4 Analytical Summary

The simulation data demonstrates that increasing pipeline depth from 1 to 12 stages yields significant performance gains. However, increasing depth to 31 stages increases the misprediction branch penalty to 30 cycles and CPI to 2.30, worsening overall execution time (0.51 ms vs 0.38 ms). Optimal pipeline depth balances clock frequency gains against hazard penalties.

QUESTION 4: Pipeline Hazard Analysis & Mitigation Techniques

4.1 Classification of Pipeline Hazards

Hazards are situations that prevent the next instruction in the instruction stream from executing during its designated clock cycle. They are grouped into three primary categories:

- **1. Structural Hazards:** Resource conflicts where hardware cannot support all simultaneous stage operations (e.g., single-ported memory accessed for IF and MEM concurrently).
- **2. Data Hazards:** Arise when an instruction depends on the result of a prior instruction that is still in flight in the pipeline.
- **3. Control Hazards:** Result from branch and jump instructions determining the Program Counter (PC) value after subsequent instructions have already entered the pipeline.

4.2 Deep Analysis of Data Hazard Mitigation

A. Data Hazards Taxonomy (RAW, WAR, WAW)

- Read-After-Write (RAW): True dependency. Instruction \$J\$ attempts to read a source register before Instruction \$I\$ writes back its result.
- Write-After-Read (WAR): Anti-dependency. Instruction \$J\$ attempts to write a destination register before Instruction \$I\$ reads its value.
- Write-After-Write (WAW): Output dependency. Instruction \$J\$ attempts to write a register before Instruction \$I\$ writes it.

B. Stalling (Bubble Insertion)

Stalling resolves hazards by inserting hardware 'bubbles' (NOPs) that freeze the PC and earlier pipeline registers while allowing dependent instructions to advance. While functionally correct, every stall cycle adds directly to overall CPI.

C. Operand Forwarding (Bypassing)

Data Forwarding adds dedicated hardware bypass paths from EX/MEM and MEM/WB pipeline registers back to the ALU inputs in the EX stage. If a RAW dependency is detected by hazard detection logic, the updated value is routed

directly without waiting for register file Write Back. (Note: Load-Use Data Hazards require a 1-cycle stall even with forwarding, because memory data is not ready until the end of the MEM stage).

4.3 Control Hazard Mitigation & Branch Prediction

Branches introduce pipeline delay because the branch outcome and target address are resolved in the EX or MEM stage.

- **Static Prediction:** Assumes all branches are either Taken or Not-Taken (e.g., predict backward branches as taken for loops, forward branches as not-taken).
- **Dynamic 2-Bit Saturating Counter:** Maintains branch history states (Strongly Taken, Weakly Taken, Weakly Not-Taken, Strongly Not-Taken). Transitions occur only after two consecutive incorrect predictions, drastically reducing branch stalls in complex loops.
- **Branch Target Buffer (BTB):** High-speed cache that stores target addresses during the Fetch stage, enabling zero-bubble target address generation.

4.4 Performance Evaluation Table

Hazard Handling Technique	Average Stalls per Instruction	Effective CPI	Performance Impact / Speedup
No Mitigation (Pure Stalling)	1.40 Stalls	2.40 CPI	Baseline Performance
Forwarding Only (Stall Load-Use)	0.15 Stalls	1.15 CPI	2.08x Speedup over Baseline
Forwarding + Static Branch Pred.	0.08 Stalls	1.08 CPI	2.22x Speedup over Baseline
Forwarding + 2-Bit Dynamic BTB	0.03 Stalls	1.03 CPI	2.33x Speedup over Baseline

QUESTION 5: Superscalar Microarchitecture, Out-of-Order Execution & SMT

5.1 Superscalar Fundamentals & Instruction Level Parallelism (ILP)

A superscalar processor achieves Instruction Level Parallelism (ILP) by fetching, decoding, issuing, and executing multiple instructions per clock cycle ($IPC > 1$). Unlike scalar processors, superscalar engines contain redundant execution units (e.g., multiple integer ALUs, vector units, load/store units, floating-point units).

5.2 Dynamic Scheduling & Out-of-Order (OoO) Execution

A. Tomasulo's Algorithm & Reservation Stations

Out-of-Order execution decouples instruction fetch/decode from execution. Instructions are fetched in program order and dispatched to Reservation Stations (buffers in front of execution units). An instruction waits in its reservation station only until its source operands become available, regardless of whether prior instructions in program order are stalled.

B. Register Renaming

False dependencies (WAR and WAW) occur due to a limited number of architectural registers. Register Renaming maps architectural registers to a larger pool of physical registers, dynamically removing WAR and WAW dependencies and allowing independent instructions to execute in parallel.

C. In-Order Commit via Reorder Buffer (ROB)

To maintain precise exception control and accurate speculative execution, state changes must occur in program order. The Reorder Buffer (ROB) holds completed instruction outputs until all prior instructions have committed. Once an instruction reaches the head of the ROB, its result writes back to architectural registers.

5.3 Simultaneous Multithreading (SMT)

Simultaneous Multithreading (SMT) extends superscalar capabilities by allowing multiple independent hardware threads to issue instructions to execution units during the exact same clock cycle. SMT converts thread-level parallelism (TLP) into ILP, effectively filling horizontal and vertical execution stalls.

5.4 Architectural Feature Comparison

Architecture Type	Instructions Fetch/Cycle	Execution Order	Hazard Management	Peak Theoretical IPC
In-Order Scalar	1	Strict In-Order	Compiler / Simple Stalls	1.0 IPC
In-Order Superscalar	2-4	Strict In-Order	Hazard stalls entire issue width	2.0-4.0 IPC
Out-of-Order Superscalar + SMT	4-8	Dynamic Out-of-Order	Tomasulo + Register Renaming + ROB	4.0-8.0 IPC

5.5 Conclusion & Summary

Superscalar organization combined with dynamic out-of-order execution, register renaming, branch prediction, and SMT represents the pinnacle of modern high-performance processor design. By maximizing resource utilization across execution units, these techniques bypass hazards and deliver superior instruction throughput.